



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE: CPE 010

SECTION: CPE21S1

DATE PERFORMED: 07-29-26

DATE SUBMITTED: 08-07-26

NAME: Paula Esguerra

INSTRUCTOR: Engr. Jimlord Quejado

SW NO. 4 Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILOA

```
main_A4.cpp
1  #include <iostream>
2  #include <stack>
3
4  void display(std::stack<int> s){
5      while (!s.empty()){
6          std::cout<<s.top() <<" ";
7          s.pop();
8      }
9      std::cout<<std::endl;
10 }
11
12 int main (){
13     std::stack<int> stack1;
14     std::stack<int> stack2;
15     std::cout << "Testing the stack STL" <<std::endl;
16
17
18     std::cout<<std::endl;
19
20     //isempty
21     std::cout << "Start up checking, is the stack empty? " << stack1.empty() << std::endl;
22
23     std::cout<<std::endl;
24
25     //push
26     stack1.push(10);
27     std::cout << "the top of the stack 1 is " <<stack1.top() <<std::endl;
28     stack1.push(9);
29     std::cout << "the top of the stack 1 is " <<stack1.top() <<std::endl;
30
31     stack2.push(8);
32     std::cout<<"the top of the stack 2 is? " << stack2.top() <<std::endl;
33
34 }
```

es Compile Log Debug Find Results

```

34     stack1.push(7);
35     std::cout<<"the top of the stack 1 is? " << stack1.top() <<std::endl;
36
37     std::cout<<std::endl;
38     std::cout<< "-----" << std::endl;
39     std::cout<<std::endl;
40
41     std::cout << "Stack 1: ";
42     display(stack1);
43
44     std::cout << "Stack 2: " ;
45     display(stack2);
46
47     std::cout<<std::endl;
48     std::cout<< "-----" << std::endl;
49     std::cout<<std::endl;
50
51     //pop
52     stack1.pop();
53     std::cout<<"Popping a number in stack 1, is the stack empty now? " <<stack1.empty() <<std::endl;
54
55     std::cout<<"Checking the size of stack 1: " <<stack1.size() <<std::endl;
56
57     std::cout<<std::endl;
58
59     stack1.pop();
60     std::cout<<"Popping a 2nd number in stack 1, is the stack empty now? " <<stack1.empty() <<std::endl;
61
62     std::cout<<"Checking the size of stack 1: " <<stack1.size() <<std::endl;
63
64     std::cout<<std::endl;
65     std::cout<< "-----" << std::endl;
66     std::cout<< "-----" << std::endl;
67     std::cout<<std::endl;
68     //emplace
69     stack1.emplace(99);
70     std::cout << "Emplace : 99 added to stack" << std::endl;
71
72     std::cout << "After Emplace Stack 1: ";
73     display(stack1);
74
75     std::cout<<std::endl;
76     std::cout<< "-----" << std::endl;
77     std::cout<<std::endl;
78     //swap
79     stack1.swap(stack2);
80     std::cout << "swap: stack 1 and stack 2 swapped successfully" << std::endl;
81
82     std::cout << "Swapped Stack 1: ";
83     display(stack1);
84
85     std::cout << "Swapped Stack 2: " ;
86     display(stack2);
87
88 }

```

C:\Users\Paramount Laptop 5

Testing the stack STL

Start up checking, is the stack empty? 1

the top of the stack 1 is 10

the top of the stack 1 is 9

the top of the stack 2 is? 8

the top of the stack 1 is? 7

Stack 1: 7 9 10

Stack 2: 8

Popping a number in stack 1, is the stack empty now? 0

Checking the size of stack 1: 2

Popping a 2nd number in stack 1, is the stack empty now? 0

Checking the size of stack 1: 1

Emplace : 99 added to stack

After Emplace Stack 1: 99 10

swap: stack 1 and stack 2 swapped successfully

Swapped Stack 1: 8

Swapped Stack 2: 99 10

Process exited after 0.4256 seconds with return value 0

Press any key to continue . . . |

ILOB

```

1  #include <iostream>
2
3  //global declaration
4  #define maxCap 10
5
6  int stackArr[maxCap];
7  int top = -1, i, newData;
8  //prototype functions
9  void push();
10 void pop();
11 void Top();
12 bool isEmpty();
13 bool isFull();
14 int choice;
15 void displayStack();
16
17 int main() {
18     //main driver
19
20     while(true)
21     {
22         std::cout << "=====\n";
23         std::cout << "Stack Operations\n";
24         std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display\n";
25         std::cout << "=====\n";
26         std::cin >> choice;
27
28         switch(choice) {
29             case 1 : push();
30             break;
31             case 2 : pop();
32             break;
33             case 3 : Top();
34             break;

```

```
34         break;
35         case 4 : std::cout << "is stack empty? " << isEmpty() << std::endl;
36         break;
37         case 5: std::cout << "is the stack full? " << isFull () <<std::endl;
38         break;
39         case 6: displayStack() ;
40         break;
41         default: std::cout<<"Invalid Choice" << std::endl;
42         break;
43     }
44 }
45 }
46
47 }
48 }
49
50 //function definition
51 bool isEmpty(){
52     if(top == -1) return true;
53     return false;
54 }
55
56 bool isFull(){
57     if(top == maxCap -1) return true;
58     return false;
59 }
60
61 void push(){
62     //error checking
63     if(isFull()){
64         std::cout << "Stack Overflow" << std::endl;
65         return;
66     }
67 }
```

```

67
68 //pushing to the stack
69 std::cout << "Enter a New Value" << std::endl;
70 std::cin >> newData;
71
72 //how do we insert the new data in the stack
73 stackArr[++top] = newData;
74
75 }
76
77 void pop() {
78     if(isEmpty()) {
79         std::cout << "Stack Underflow" << std::endl;
80         return;
81     }
82
83     std::cout << "Popping: " << stackArr[top] << std::endl;
84     top--;
85 }
86
87 void Top() {
88     //error catching
89     if(isEmpty()) {
90         std::cout << "The stack is Empty" << std::endl;
91         return;
92     }
93     //check the top value:
94     std::cout << "top element: " << stackArr[top] << std::endl;
95 }
96
97
98 void displayStack() {
99     if (isEmpty()) {
100         std::cout << "Stack is Empty" << std::endl;
101         return;
102     }
103     for (i=top; i>=0; i--) {
104         std::cout << stackArr[i] << std::endl;
105     }
106 }
107
108

```

```
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
2
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
3
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
4
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
5
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
6
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
```

```

1
Enter a New Value
6
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
7
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
8
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
9
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
2
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
1
Enter a New Value
3
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

```


Enter a New Value

3

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

1

Stack Overflow

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

1

Stack Overflow

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

6

3

2

9

8

7

6

5

4

3

2

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

5

is the stack full? 1

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

4

is stack empty? 0

```
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
3
top element: 3
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
Popping: 3
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
Popping: 2
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
Popping: 9
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
Popping: 8
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
Popping: 7
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
2
```

Popping: 6

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Popping: 5

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Popping: 4

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Popping: 3

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Popping: 2

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Stack Underflow

=====

Stack Operations

1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display

=====

2

Stack Underflow

=====

Stack Operations

```
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
3
The stack is Empty
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
4
is stack empty? 1
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
5
is the stack full? 0
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
6
Stack is Empty
=====
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display
=====
|
```

CONCLUSION

I don't get the whole code yet, but I understand how it works. In ILOA we implemented std::stack, LIFO, Last in First out, where, for example, if I added 4, 5, 6 respectively, 4 will be on the last one on the list, and 6 will be the first one, so if I pop (pop is where I remove the number) a number, 6 will be the first one to be deleted. Then emplace is where I could add a number, and maybe if I desired it, I could store it in between two numbers, but for now I generally put it on top of the stack, so if I put 30, then after 6, 30 will be the first on the top and to be popped. Swap can swap the stored functions in a stack, so for example, I have stack 1, which has 4,5,6, 30 respectively, and stack 2 that has 7 and 0, so if I use the swap function, the elements that stack 1 will be exchanged with stack 2

and vice versa. Then on ILOB, Top function is where I could check who or what element is on the top, which was the last one input to. isEmpty is to check if a certain stack has no values inside, then it will answer true and false, as also to isFull. Lastly the display function, it is a bit tricky for me, because I'm not still not used using the i= or i<, I get how it functions and I could read it a bit, but not fully, so I had the help of search it in google and asking gemini for clarifications, for that this is the end of my conclusion.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOC(Supplementary Activity)

```

1  #include <iostream>
2  #include <string>
3
4  #define maxCap 100
5
6  char stackArr[maxCap];
7  int top = -1;
8
9  bool isEmpty() {
10     return top == -1;
11 }
12
13 bool isFull() {
14     return top == maxCap - 1;
15 }
16
17 void push(char newData) {
18     if (isFull()) {
19         std::cout << "Stack Overflow" << std::endl;
20         return;
21     }
22     stackArr[++top] = newData; // position first before writing the character
23 }
24
25 void pop() {
26     if (isEmpty()) {
27         std::cout << "Stack Underflow" << std::endl;
28         return;
29     }
30     top--; // decrements the index down by 1
31 }
32
33 char Top() {
34     if (isEmpty()) {
35         return '\0'; // prevent viewing random garbage memory
36     }
37     return stackArr[top];
38 }

```

```

39
40 // Balance checking driver using the global stack
41 bool checkBalance(std::string expr) {
42     // Reset global stack pointer for each new test string
43     top = -1;
44
45     for (int i = 0; i < expr.length(); i++) {
46         char ch = expr[i];
47
48         // Step 2a: Ignore non-bracket characters
49         if (ch != '(' && ch != '{' && ch != '[' && ch != ')' && ch != '}' && ch != ']') {
50             continue;
51         }
52
53         // Step 2b: Push opening symbols
54         if (ch == '(' || ch == '{' || ch == '[') {
55             push(ch);
56         }
57
58         // Step 2c: Handle closing symbols
59         else if (ch == ')' || ch == '}' || ch == ']') {
60             if (isEmpty()) {
61                 return false; // Error: closing symbol with no opening partner
62             }
63
64             char open = Top();
65             pop();
66
67             // Step 2d: Verify matching pairs
68             if ((ch == ')' && open != '(') ||
69                 (ch == '}' && open != '{') ||
70                 (ch == ']' && open != '[')) {
71                 return false; // Error: mismatched bracket types
72             }
73         }
74     }
75
76     // Step 3: If stack is empty, it is balanced
77     return isEmpty();
78 }
79
80 int main() {
81     // The exact expressions from your assignment table image
82     std::string expressions[] = {
83         "(A+B)+(C-D)",
84         "((A+B)+(C-D))",
85         "((A+B)+[C-D])",
86         "((A+B)+[C-D])",
87     };
88
89     std::cout << "--- Testing Array Global Stack ---\n" << std::endl;
90
91     for (int i = 0; i < 4; i++) {
92         std::string currentExpr = expressions[i];
93
94         std::cout << "Expression: " << currentExpr << std::endl;
95
96         if (checkBalance(currentExpr)) {
97             std::cout << "Result: Valid (Y)\n" << std::endl;
98         } else {
99             std::cout << "Result: Invalid (N)\n" << std::endl;
100         }
101         std::cout << "-----\n" << std::endl;
102     }
103
104     return 0;

```

OUTPUT:

```
C:\Users\Paramount Laptop 5 × + v
--- Testing Array Global Stack ---

Expression: (A+B)+(C-D)
Result: Valid (Y)

-----

Expression: ((A+B)+(C-D)
Result: Invalid (N)

-----

Expression: ((A+B)+[C-D])
Result: Valid (Y)

-----

Expression: ((A+B)+[C-D]}
Result: Invalid (N)

-----

-----

Process exited after 0.3261 seconds with return value 0
Press any key to continue . . .
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
$(A+B)+(C-D)$	Y	Expression: $(A+B)+(C-D)$ Result: Valid (Y)	The parentheses are paired up, balanced.
$((A+B)+(C-D)$	N	Expression: $((A+B)+(C-D)$ Result: Invalid (N)	There are missing pair after the C-D parentheses, leaving it unbalanced.
$((A+B)+[C-D])$	Y	Expression: $((A+B)+[C-D])$ Result: Valid (Y)	The braces and brackets are completely paired that is why it is valid and balanced.
$((A+B)+[C-D])\}$	N	Expression: $((A+B)+[C-D])\}$ Result: Invalid (N)	This expression is invalid because there was a mismatch after and before the opening and closing of the expression.

TOOLS ANALYSIS

How do different internal representations affect that implementation and usage of the stack?

- The array representation uses fixed, contiguous memory, the implementation of array representation uses straightforward elements that are accessed in an index tracker, like `top()`, however, it can be constrained using `maxcap()`.
- The linked list representation uses dynamic memory allocation, where each element is and linked by a pointer, the implementation requires a manual pointer manipulation and a destructor to prevent memory leaks. The stack size can grow or shrinks.
- STL stacks is highly reliable, safe and fully optimized, it requires no underlying code maintenance from the programmer.

CONCLUSION

My conclusion in this activity, I get how it works, but I still need practice to independently do the code. I have learned that LIFO concept is for data processing through stacks operation, where the last element to go in, will be the first element to go out, or will be declared as the first not as the last element. The push and pop decisions are opening symbols that can immediately stages onto the stack track line - `pop()`; `top()`. Core logical behaviors like push, pop, isEmpty and etc.. can remain uniform even different data types are managed. If I improve improve maybe I can do more complex coding using the different types of implementation, then I can use more pointer operators and memory management. This concludes my Conclusion.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT



COMPUTER
ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Error assistance, and asking questions

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I pasted my work to check something and if there were an error, I will retype and try to understand the changed code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.